Experiment 17

Banker's Algorithm

code:

```
#include <stdio.h>

#define P 5
#define R 3

int available[R] = {3, 3, 2};

int maximum[P][R] = {
  {7, 5, 3},
  {3, 2, 2},
  {9, 0, 2},
  {2, 2, 2},
  {4, 3, 3}
};

int allocation[P][R] = {
  {0, 1, 0},
  {2, 0, 0},
  {3, 0, 2},
  {2, 1, 1},
  {0, 0, 2}
};

int need[P][R];

int isSafe() {
  int work[R], finish[P] = {0};
  int safeSequence[P];
```

```
int count = 0;

for (int i = 0; i < R; i++)
   work[i] = available[i];

while (count < P) {
   int found = 0;

   for (int i = 0; i < P; i++) {

      if (finish[i] == 0) {
         int j;

         for (j = 0; j < R; j++) {
            if (need[i][j] > work[j])
               break;
         }

         if (j == R) {

            for (int k = 0; k < R; k++)
               work[k] += allocation[i][k];

            safeSequence[count++] = i;
            finish[i] = 1;
            found = 1;
         }
      }
   }

   if (found == 0)
```

```
            return 0;
        }

        printf("Safe Sequence: ");

        for (int i = 0; i < P; i++)
            printf("P%d ", safeSequence[i]);

        printf("\n");

        return 1;
    }

    int main() {
        int request[R];
        int process;

        for (int i = 0; i < P; i++) {
            for (int j = 0; j < R; j++) {
                need[i][j] = maximum[i][j] - allocation[i][j];
            }
        }

        printf("Available Resources: ");
        for (int i = 0; i < R; i++)
            printf("%d ", available[i]);

        printf("\n\nEnter Process Number (0-4): ");
        scanf("%d", &process);

        if (process < 0 || process >= P) {
```

```
    printf("Invalid process number.\n");
    return 0;
}

printf("Enter Resource Request: ");

for (int i = 0; i < R; i++)
    scanf("%d", &request[i]);

for (int i = 0; i < R; i++) {

    if (request[i] < 0 ||
        request[i] > need[process][i]) {
        printf("Request exceeds maximum need.\n");
        return 0;
    }

    if (request[i] > available[i]) {
        printf("Resources not available. Process must wait.\n");
        return 0;
    }
}

for (int i = 0; i < R; i++) {
    available[i] -= request[i];
    allocation[process][i] += request[i];
    need[process][i] -= request[i];
}

if (isSafe()) {
    printf("Request granted.\n");
```

```
    } else {

        for (int i = 0; i < R; i++) {
            available[i] += request[i];
            allocation[process][i] -= request[i];
            need[process][i] += request[i];
        }

        printf("Request denied. System is unsafe.\n");
    }

    return 0;
}
```

Output:

